Estructura de datos

Árboles AVL

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2026-II

UACM
Universidad Autónoma de la Ciudad de México
NADA HUMANO ME ES AJENO

Índice

1 Fundamentos

2 Rotaciones

3 Implementación de un AVL

4 Algoritmos de balanceo

5 Operaciones del AVL

6 Recorridos y uso

Del ABB al AVL

- Un **ABB ordena:** izquierda menor, derecha mayor.
- Insertar **datos ya ordenados** puede producir una cadena.
- Un AVL es un **ABB autoequilibrado:** controla la diferencia de alturas.
- Las rotaciones **preservan** el orden de búsqueda.

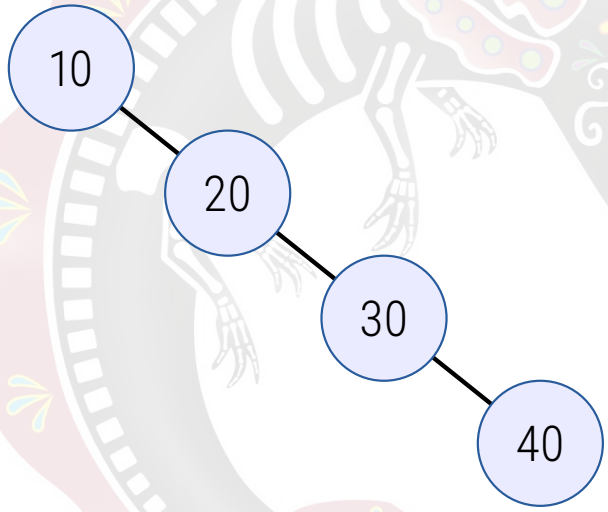


ABB degenerado

Altura de un nodo

Convención usada en la implementación

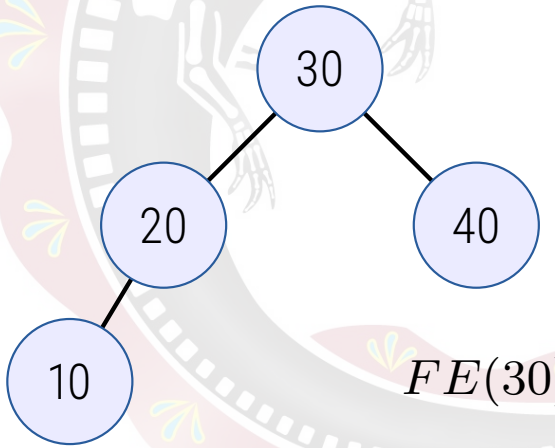
$$h(n) = \begin{cases} -1, & n = \texttt{null} \\ 1 + \max(h(n.izq), h(n.der)), & \text{otro caso} \end{cases}$$

- **Hoja:** altura 0.
- **Árbol vacío:** altura -1.
- La altura **se almacena** en cada `NodoArbol<E>` y **se actualiza** después de cambios estructurales.

Factor de equilibrio (FE)

$$FE(n) = h(n.izquierdo) - h(n.derecho)$$

- $FE = +1$: izquierda un nivel más alta.
- $FE = 0$: alturas iguales.
- $FE = -1$: derecha un nivel más alta.
- Si $|FE| > 1$, hay desbalance.



$FE(30) = 1$

Invariante del AVL

Dos propiedades deben cumplirse simultáneamente

1. **Orden de ABB:** todos los valores del subárbol izquierdo son **menores** y los del derecho **mayores.**
2. **Equilibrio AVL:** para todo nodo n, $FE(n) \in \{-1, 0, 1\}$.

Importante

Un árbol AVL **no tiene** por qué ser **perfecto ni completo:** basta con satisfacer **ambas propiedades** en todos los nodos.

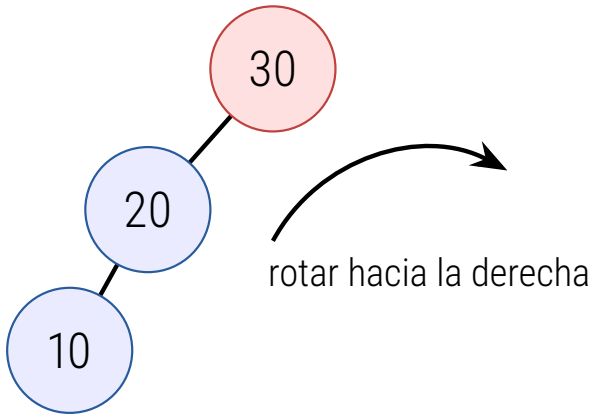
Cuatro situaciones de desbalance

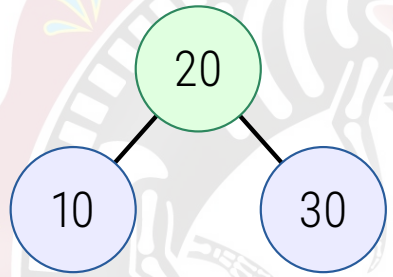
Caso	Señal del desbalance	Corrección
LL	izquierda del hijo izquierdo	rotación derecha
RR	derecha del hijo derecho	rotación izquierda
LR	derecha del hijo izquierdo	izquierda y derecha
RL	izquierda del hijo derecho	derecha e izquierda

Regla práctica

Se inspecciona el **nodo desbalanceado** y el **factor de equilibrio** de su hijo más alto.

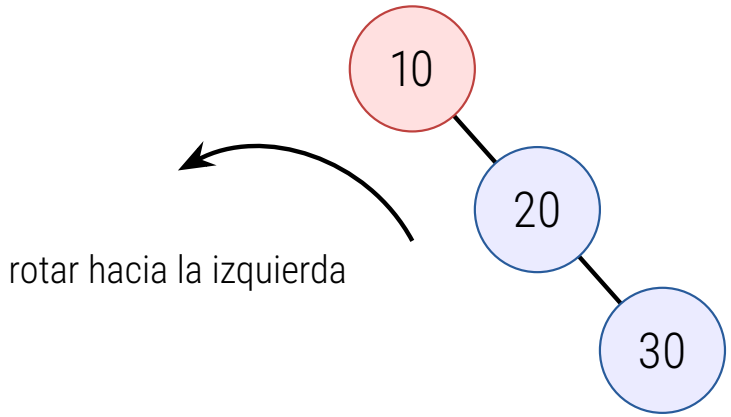
Caso LL: rotación simple a la derecha

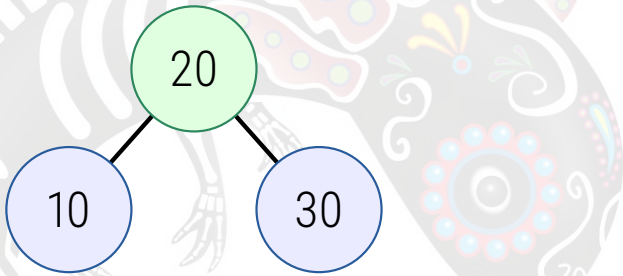




- **Insertar:** 30, 20, 10.
- La **nueva raíz** del subárbol es 20.

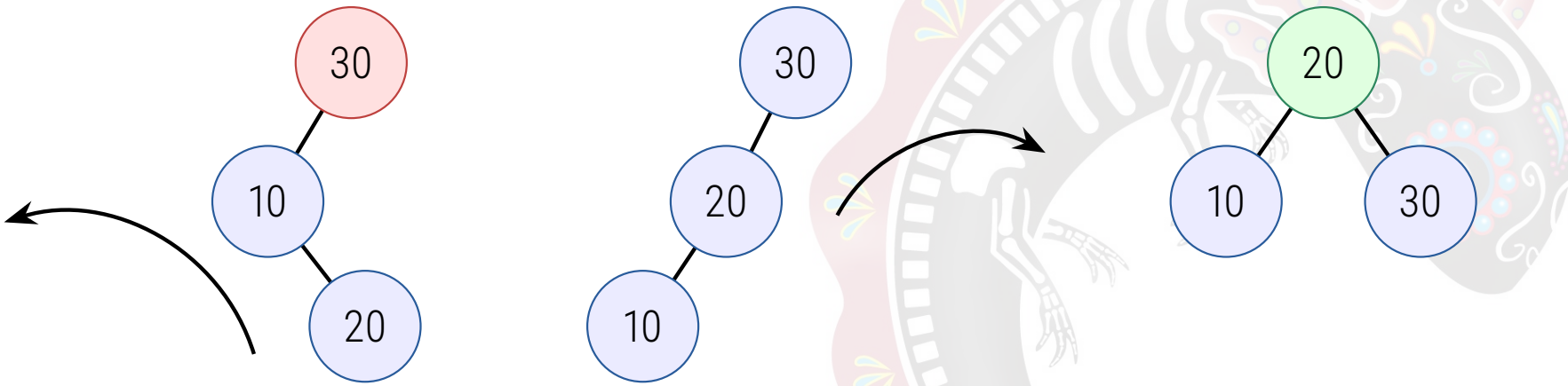
Caso RR: rotación simple a la izquierda





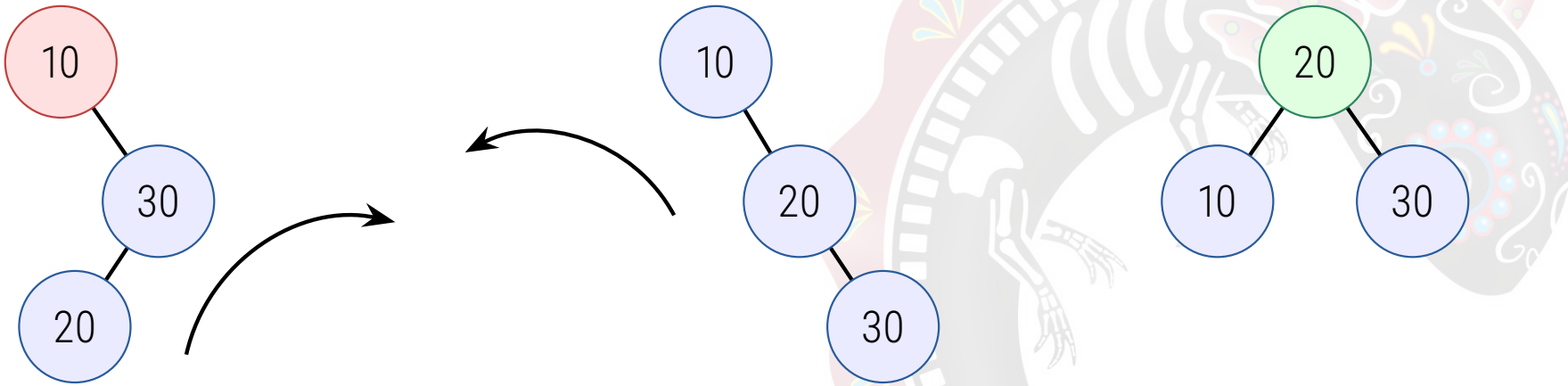
- **Insertar:** 10, 20, 30.
- La **nueva raíz** del subárbol es 20.

Caso LR: rotación doble izquierda-derecha



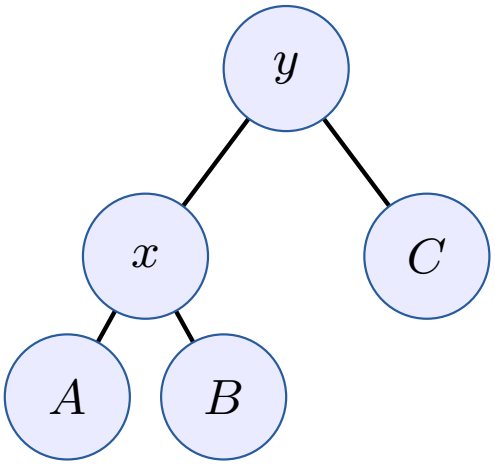
1. Rotar **a la izquierda** sobre 10.
2. Rotar **a la derecha** sobre 30.

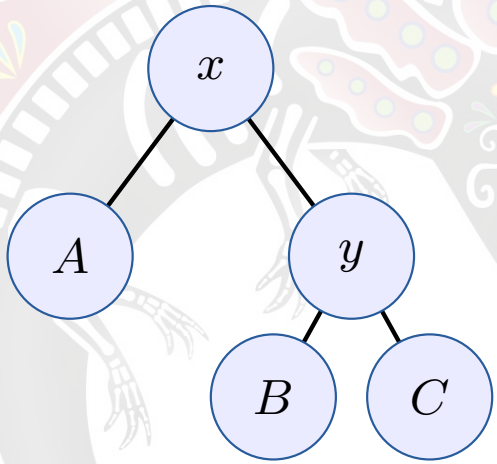
Caso RL: rotación doble derecha-izquierda



1. Rotar **a la derecha** sobre 30.
2. Rotar **a la izquierda** sobre 10.

Las rotaciones conservan el orden





$$A < x < B < y < C$$

En una **rotación derecha,** el subárbol B cambia de padre, pero **conserva** su posición relativa.

Interfaz e implementación

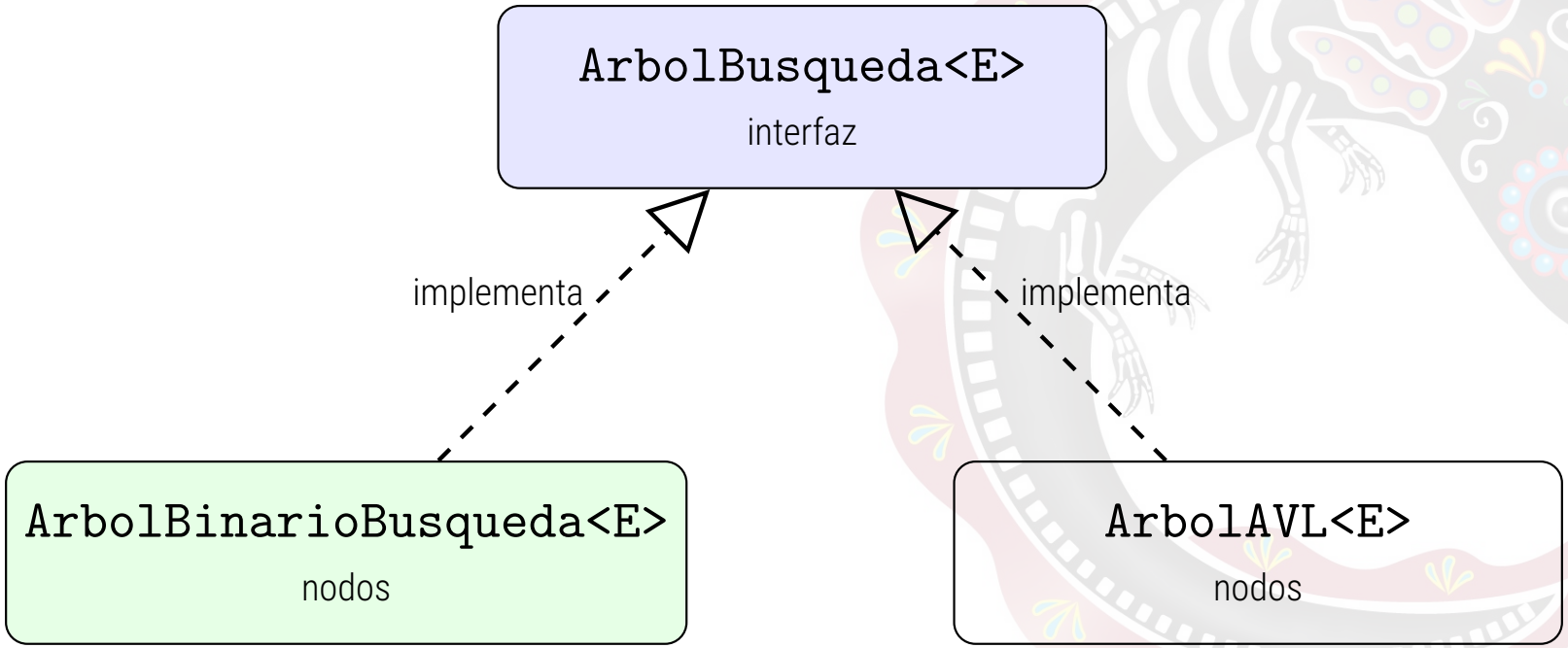
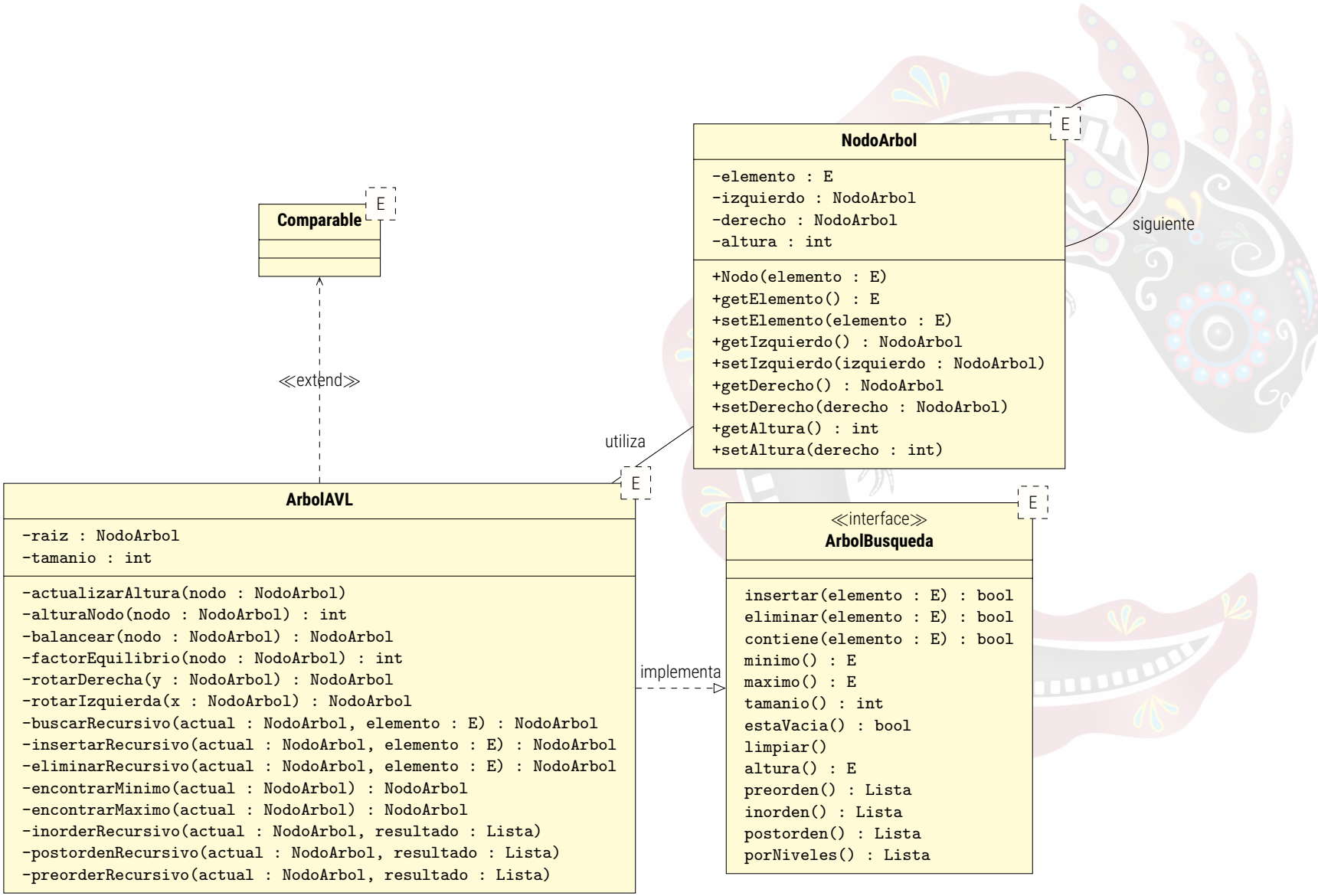


Diagrama de clases



NodoArbol: estructura y altura

```
public class NodoArbol<E> {
  private E elemento;
  private NodoArbol<E> izquierdo;
  private NodoArbol<E> derecho;
  private int altura;

  public NodoArbol(E elemento) {
    this.elemento = elemento;
    this.altura = 0; // nuevo nodo hoja
  }
  // getters y setters de los cuatro atributos
}
```

Diferencia frente al ABB ordinario

El atributo `altura` se **mantiene actualizado** en el AVL. El ABB previo puede seguir calculando su altura recursivamente.

ArbolAVL: estado y métodos auxiliares

```
public class ArbolAVL<E extends Comparable<E>>
implements ArbolBusqueda<E> {
  private NodoArbol<E> raiz;
  private int tamanio;

  private int alturaNodo(NodoArbol<E> n) {
    return n == null ? -1 : n.getAltura();
  }
  private int factorEquilibrio(NodoArbol<E> n) {
    return n == null ? 0 :
    alturaNodo(n.getIzquierdo())
    - alturaNodo(n.getDerecho());
  }
}
```

Pseudocódigo: actualizar altura

Algoritmo 1 ActualizarAltura(n)

1: **if** $n = \texttt{null}$ **then**
2: **return**
3: **end if**
4: $h_i \leftarrow$ ALTURANODO$(n.izquierdo)$
5: $h_d \leftarrow$ ALTURANODO$(n.derecho)$
6: $n.altura \leftarrow 1 + \max(h_i, h_d)$

Se actualiza desde los nodos inferiores hacia la raíz al regresar de la recursión.

Pseudocódigo: rotación derecha

Algoritmo 2 RotarDerecha(y)

1: $x \leftarrow y.izquierdo$
2: $B \leftarrow x.derecho$
3: $x.derecho \leftarrow y$
4: $y.izquierdo \leftarrow B$
5: ACTUALIZARALTURA(y)
6: ACTUALIZARALTURA(x)
7: **return** x

Primero se actualiza la altura del antiguo padre y; después la de la nueva raíz x.

Pseudocódigo: rotación izquierda

Algoritmo 3 RotarIzquierda(x)

1: $y \leftarrow x.derecho$
2: $B \leftarrow y.izquierdo$
3: $y.izquierdo \leftarrow x$
4: $x.derecho \leftarrow B$
5: ACTUALIZARALTURA(x)
6: ACTUALIZARALTURA(y)
7: **return** y

El método devuelve la nueva raíz del subárbol; quien lo invoca debe actualizar su referencia.

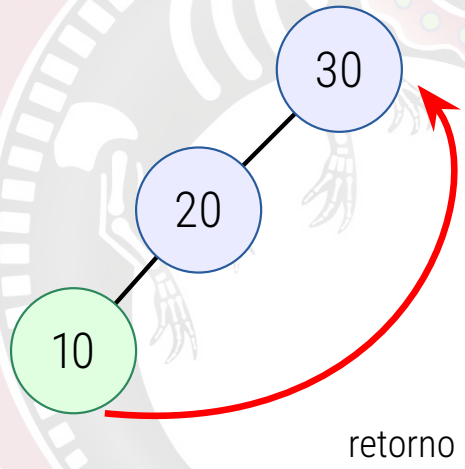
Pseudocódigo: balancear un subárbol

Algoritmo 4 Balancear(n)

```
1:  if n = null then
2:      return null
3:  end if
4:  ActualizarAltura(n)
5:  f ← FactorEquilibrio(n)
6:  if f > 1 then
7:      if FactorEquilibrio(n.izquierdo) < 0 then
8:          n.izquierdo ← RotarIzquierda(n.izquierdo)
9:      end if
10:     return RotarDerecha(n)
11: else if f < −1 then
12:     if FactorEquilibrio(n.derecho) > 0 then
13:         n.derecho ← RotarDerecha(n.derecho)
14:     end if
15:     return RotarIzquierda(n)
16: end if
17: return n
```

¿Por qué el balanceo ocurre al regresar?

1. Descender hasta encontrar la posición de inserción.
2. Crear el nuevo nodo.
3. Regresar por la pila recursiva.
4. Actualizar alturas y balancear cada ancestro afectado.



Pseudocódigo: inserción pública

Algoritmo 5 Insertar(elemento)

1: **if** $elemento = \texttt{null}$ **then**
2: **error**
3: **end if**
4: **if** CONTIENE(elemento) **then**
5: **return** falso
6: **end if**
7: $raiz \leftarrow$ INSERTARREC$(raiz, elemento)$
8: $tamanio \leftarrow tamanio + 1$
9: **return** verdadero

La versión didáctica evita duplicados con una búsqueda previa.

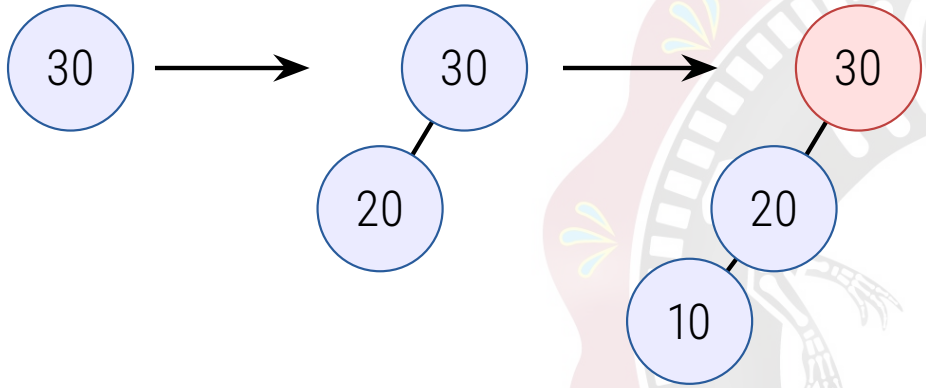
Pseudocódigo: inserción recursiva

Algoritmo 6 InsertarRec(n,elemento)

1: **if** $n =$ `null` **then**
2: **return** nuevo `NodoArbol(elemento)`
3: **end if**
4: **if** $elemento < n.elemento$ **then**
5: $n.izquierdo \leftarrow$ INSERTARREC$(n.izquierdo, elemento)$
6: **else if** $elemento > n.elemento$ **then**
7: $n.derecho \leftarrow$ INSERTARREC$(n.derecho, elemento)$
8: **end if**
9: **return** BALANCEAR(n)

En Java, las comparaciones se realizan con `elemento.compareTo(n.getElemento())`.

Inserción paso a paso: 30, 20, 10



Resultado

El nodo 30 queda con $FE = +2$. Se reconoce LL y se aplica una rotación derecha: la raíz del subárbol pasa a ser 20.

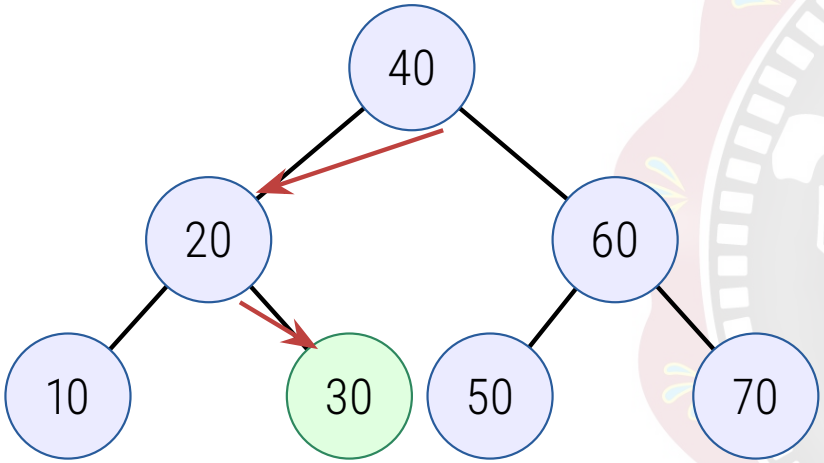
Pseudocódigo: búsqueda iterativa

Algoritmo 7 Contiene(elemento)

```
1:  actual ← raiz
2:  while actual ≠ null do
3:      if elemento = actual.elemento then
4:          return verdadero
5:      else if elemento < actual.elemento then
6:          actual ← actual.izquierdo
7:      else
8:          actual ← actual.derecho
9:      end if
10: end while
11: return falso
```

La búsqueda en AVL es igual a la de un ABB; no se realizan rotaciones.

Búsqueda: seguir una sola rama



Buscar 30

Comparaciones: $30 < 40; 30 > 20; 30 = 30.$

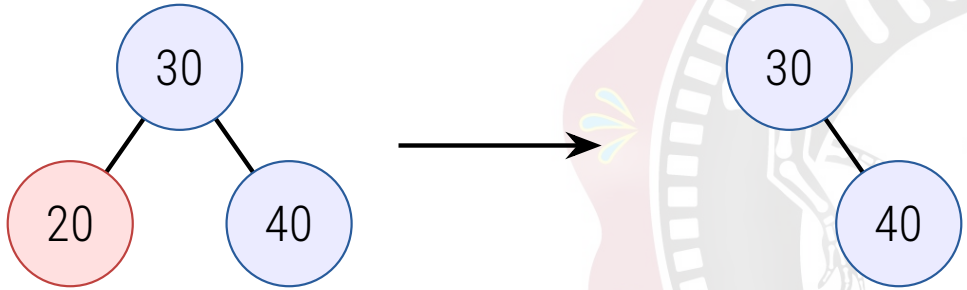
Eliminación: tres casos del ABB

1. **Hoja:** se elimina devolviendo `null`.
2. **Un hijo:** el hijo sustituye al nodo eliminado.
3. **Dos hijos:** se reemplaza por el sucesor inorden (mínimo del subárbol derecho) y se elimina ese sucesor.

Diferencia AVL

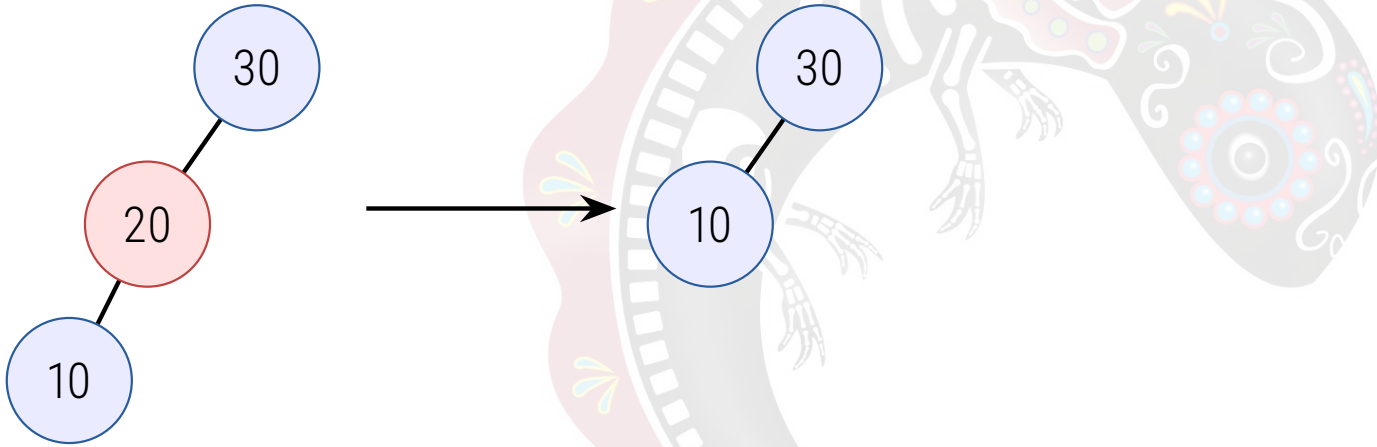
Al regresar de la eliminación, se actualizan alturas y se balancean todos los ancestros afectados.

Eliminación: nodo hoja



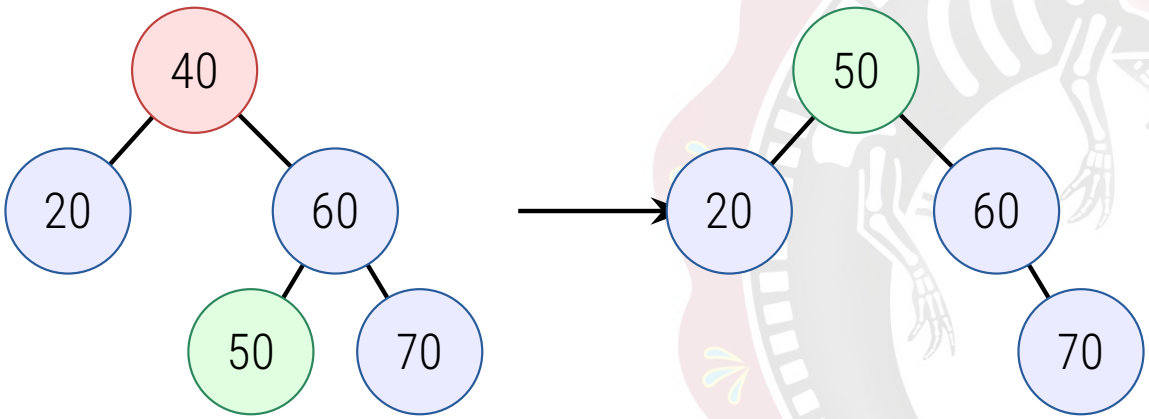
Se elimina 20; después se recalcula la altura de 30 y se verifica su equilibrio.

Eliminación: nodo con un hijo



El hijo 10 reemplaza a 20. El ancestro 30 se actualiza y, si corresponde, se balancea.

Eliminación: nodo con dos hijos



Para eliminar 40, copiar el sucesor 50 y eliminar el nodo que contenía 50; luego balancear.

Pseudocódigo: eliminación pública

Algoritmo 8 Eliminar(elemento)

1: **if** $elemento = \texttt{null}$ **then**
2: **error**
3: **end if**
4: **if no** CONTIENE(elemento) **then**
5: **return** falso
6: **end if**
7: $raiz \leftarrow$ ELIMINARREC$(raiz, elemento)$
8: $tamanio \leftarrow tamanio - 1$
9: **return** verdadero

La comprobación previa evita disminuir el tamaño cuando el elemento no existe.

Pseudocódigo: eliminación recursiva I

Algoritmo 9 EliminarRec(n,elemento)

```
1:  procedure ELIMINARREC(n,elemento)
2:      if n = null then
3:          return null
4:      end if
5:      if elemento < n.elemento then
6:          n.izquierdo ← ELIMINARREC(n.izquierdo, elemento)
7:      else if elemento > n.elemento then
8:          n.derecho ← ELIMINARREC(n.derecho, elemento)
9:      else
10:         if n.izquierdo = null then
11:             return n.derecho
12:         end if
13:         if n.derecho = null then
```

Pseudocódigo: eliminación recursiva II

```
14:             return n.izquierdo
15:         end if
16:         sucesor ← EncontrarMinimo(n.derecho)
17:         n.elemento ← sucesor.elemento
18:         n.derecho ← EliminarRec(n.derecho, sucesor.elemento)
19:         return Balancear(n)
20:     end if
21: end procedure
```

Al regresar de cada rama

También se ejecuta `return balancear(n)` después de los casos en que el elemento buscado está en un subárbol izquierdo o derecho.

Eliminar puede exigir nuevas rotaciones

- La altura de un subárbol puede disminuir después de una eliminación.
- Ese cambio puede desbalancear varios ancestros sucesivos.
- Por eso `EliminarRec` llama a `Balancear` durante el retorno recursivo.
- En eliminación son posibles casos donde el hijo más alto tiene $FE = 0$; las condiciones `<0` y `>0` de `Balancear` los tratan con rotación simple.

Recorridos: resultado de tipo `Lista<E>`

Método	Orden
`preorden()`	raíz, izquierda, derecha
`inorden()`	izquierda, raíz, derecha
`postorden()`	izquierda, derecha, raíz
`porNiveles()`	por profundidad creciente, de izquierda a derecha

Decisión de diseño

El recorrido no imprime datos: construye una `ListaArreglos<E>` y la devuelve.
El código cliente decide cómo mostrarla o procesarla.

Pseudocódigo: inorden

Algoritmo 10 Inorden()

1: $resultado \leftarrow$ nueva `ListaArreglos<E>`
2: INORDENREC(raiz,resultado)
3: **return** $resultado$

Algoritmo 11 InordenRec(n,resultado)

1: **if** $n =$ `null` **then**
2: **return**
3: **end if**
4: INORDENREC(n.izquierdo,resultado)
5: AGREGAR(resultado,n.elemento)
6: INORDENREC(n.derecho,resultado)

Preorden y postorden: cambiar la visita

Preorden

```
1: procedure PreRec(n, L)
2:     if n = null then
3:         return
4:     end if
5:     Agregar(L, n.elemento)
6:     PreRec(n.izquierdo, L)
7:     PreRec(n.derecho, L)
8: end procedure
```

Postorden

```
1: procedure PostRec(n, L)
2:     if n = null then
3:         return
4:     end if
5:     PostRec(n.izquierdo, L)
6:     PostRec(n.derecho, L)
7:     Agregar(L, n.elemento)
8: end procedure
```

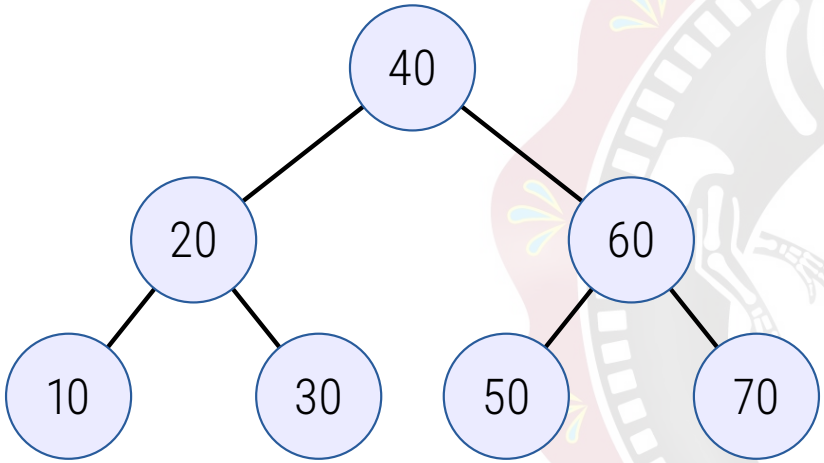
Los métodos públicos crean una lista, invocan al auxiliar con la raíz y devuelven la lista.

Pseudocódigo: recorrido por niveles

Algoritmo 12 PorNiveles()

```
1:  resultado ← nueva ListaArreglos<E>
2:  if raiz = null then
3:      return resultado
4:  end if
5:  cola ← nueva ColaEnlazada<NodoArbol<E>>
6:  ENCOLAR(cola,raiz)
7:  while no ESTAVACIA(cola) do
8:      actual ← DESENCOLAR(cola)
9:      AGREGAR(resultado,actual.elemento)
10:     if actual.izquierdo ≠ null then
11:         ENCOLAR(cola,actual.izquierdo)
12:     end if
13:     if actual.derecho ≠ null then
14:         ENCOLAR(cola,actual.derecho)
15:     end if
16: end while
17: return resultado
```

Recorrido por niveles: estado de la cola



Al desencolar 40, se encolan 20 y 60. Después se procesan 20, 60, 10, 30, 50 y 70.
Resultado: 40, 20, 60, 10, 30, 50, 70.

Complejidad de las operaciones

Operación	ABB (peor caso)	AVL (peor caso)
Búsqueda	$O(n)$	$O(\log n)$
Inserción	$O(n)$	$O(\log n)$
Eliminación	$O(n)$	$O(\log n)$
Altura del árbol	$O(n)$	$O(\log n)$
Recorrido completo	$O(n)$	$O(n)$

Las rotaciones individuales cuestan $O(1)$; mantener la altura en cada nodo permite calcular el FE en tiempo constante. El método `altura()` del AVL implementado cuesta $O(1)$ porque lee la altura almacenada en la raíz.

Fin de la presentación



¿Preguntas o comentarios?